

Piping from rg to llm to answer questions about code

Here's a trick I've used a couple of times in the past few days.

If I have a question about a codebase and I want to answer it using an LLM, I'll use `ripgrep` (aka `rg`) to search that codebase for a specific symbol and output the 5-10 lines of code that follow that symbol.

Then I'll pipe the result to my [llm CLI tool](#), running through a large context model such as Claude or `gpt-4-turbo`, and ask it a question about the code.

The question

I wanted to figure out what "read ribbons" are in the source code for <https://lobste.rs/> - which I checked out from GitHub like this:

```
cd /tmp
git clone https://github.com/lobsters/lobsters
cd lobsters
```

The recipe

Here's the recipe I use, explained in full below:

```
rg -I -A 5 read_ribbons | \
  llm --system 'explain what read ribbons are' -m gpt-4-turbo
```

Which gave me:

Based on the SQL queries and the schema definitions you've provided, it seems "read ribbons" is a concept related to tracking user interactions with stories or posts in a social platform or content management system, possibly a forum, news aggregator, or a blog commenting system.

Here's an explanation of key components related to read ribbons and their functionality as inferred from the provided SQL snippets:

1. **Purpose:** Read ribbons seem to serve the purpose of tracking whether users have read or followed specific stories or posts. This feature is likely used to manage and display unread content to users, enhancing their experience by making it easier to catch up on what's new or changed since their last visit.
2. **Database Structure:**
 - The `read_ribbons` table has a few important fields: `is_following` (a boolean indicating if a user is following a story), `created_at` and

updated_at (timestamps for tracking when the read ribbon was created or last updated), user_id, and story_id (foreign keys linking to the users and stories tables respectively). It essentially creates a many-to-many relationship between users and stories, with the additional context about following status and read status.

- There are indices on user_id and story_id for performance optimization, ensuring that queries to find which stories a user is following (or who is following a story) are efficient.

3. **Functionality:**

- Queries involving the read_ribbons table seem to be focused on fetching unread comments for the user, distinguishing between unread and read comments based on the updated_at timestamp in comparison to the created_at timestamp of comments.
- The concept of "following" a story is important here. A user appears to automatically follow stories they interact with (e.g., through commenting), and their read ribbons are updated when they read new comments to ensure accurate tracking of what they've seen.
- Additional conditions in the queries include checks for comment and story scores (to filter out negatively scored items), comment moderation status, and ensuring that users do not see their comments as unread.

4. **Actions:** The operations performed with read ribbons include updating the updated_at timestamp when a user views new content, and filtering comments and stories to present users with relevant, unread discussions. Additionally, after certain actions like viewing unread replies, corresponding "after actions" (such as update_read_ribbons) are triggered to keep the read ribbon statuses up to date.

To sum up, read ribbons are a feature used for managing and tracking user engagement with stories or posts, distinguishing between read and unread content to improve user experience on platforms where content is frequently updated or added.

rg -IN -A 5 searchterm

[ripgrep](#) is a lightning fast grep-style tool. If you give it a search term it will find that term in every file (including in subdirectories) of your current directory:

```
rg read_ribbons
```

```
db/views/replying_comments_v01.sql
2:   read_ribbons.user_id,
4:   read_ribbons.story_id as story_id,
10:  (read_ribbons.updated_at < comments.created_at) as is_unread
12:  read_ribbons
14:  comments ON comments.story_id = read_ribbons.story_id
```

```

20:   read_ribbons.is_following = 1
21:   AND comments.user_id != read_ribbons.user_id
23:     (parent_comments.user_id = read_ribbons.user_id
25:       AND stories.user_id = read_ribbons.user_id))

db/schema.rb
177: create_table "read_ribbons", id: { type: :bigint, unsigned: true
}, charset: "utf8mb4", collation: "utf8mb4_general_ci", force: :cascade
do |t|
183:   t.index ["story_id"], name: "index_read_ribbons_on_story_id"
184:   t.index ["user_id"], name: "index_read_ribbons_on_user_id"
...

```

For the purposes of feeding code into a large language model those line numbers are just wasted tokens. Using `-N` removes them:

```
rg -N read_ribbons
```

```

db/views/replying_comments_v01.sql
  read_ribbons.user_id,
  read_ribbons.story_id as story_id,
...

```

The first time I wrote up this tip I had a nasty shock when I found that piping content from `rg` outputs it in a different format to if you send it to the CLI. That's illustrated by this:

```
rg -N read_ribbons | cat
```

```

db/views/replying_comments_v01.sql:   read_ribbons.user_id,
db/views/replying_comments_v01.sql:   read_ribbons.story_id as story_id,
db/views/replying_comments_v01.sql:   (read_ribbons.updated_at <
comments.created_at) as is_unread
db/views/replying_comments_v01.sql:   read_ribbons
...

```

Those filenames are even more wasted tokens! Adding the `-I/--no-filename` option fixes that problem:

```
rg -IN read_ribbons | cat
```

And it turns out `rg -I` has the same result as `rg -IN`, at least for piped content (checked using `| cat`).

I want a bit more context. I've found that for this purpose returning the 5 or 10 lines following a match works great, which can be achieved using `-A 5` (for `--after-context`). `rg` will merge together overlapping sections:

```
rg -I -A 5 read_ribbons
```

That produces output like this - no line numbers, no filenames, and a -- between each match:

```
...
--
  after_action :update_read_ribbons, only: [:unread]
  after_action :clear_unread_replies_cache, only: [:comments, :stories]
  after_action :zero_unread_replies_cache, only: [:all, :unread]

  def all
    @title = "All Your Replies"
  --
  def update_read_ribbons
    story_ids = @replies.pluck(:story_id).uniq
    ReadRibbon
      .where(user_id: @user.id, story_id: story_ids)
      .update_all(updated_at: Time.current)
  end
```

Tokens cost money, so I ran a check to see the token count by piping through my [ttok](#) tool:

```
rg -I -A 5 read_ribbons | ttok
```

It replied 4312 which is fine - GPT-4 Turbo [is currently priced](#) at 1c per 1,000 input tokens.

The -C 5 gives 5 lines before the match and 5 lines after. -B 5 is just 5 lines before.

Piping to llm

My llm tool accepts piped content, and can also take a --system prompt (for models that support it) providing instructions about what to do with that content. Hence this final recipe:

```
rg -I -A 5 read_ribbons | \
  llm --system 'explain what read ribbons are' -m gpt-4-turbo
```

The -m gpt-4-turbo option can be shortened to -m 4t.

You can see the [full prompt and response here](#), extracted using `llm logs -c | pbcopy`.